

Terminology

Features

The columns other than the Target column. These columns help the ML model to predict the target for future data points.

Target Variable

The column that we want to predict. It is our resultant column and we want to know for future data.

Clustering

A way of grouping the data points into different clusters, consisting of similar data points. Objects with possible similarities remain in a group that has less or no similarities with another group

Naive Bayes

Naive Bayes is called **naive** because it makes a strong, often unrealistic, assumption that all predictor features are independent of each other and contribute equally to the outcome. But in real-world data, features are often correlated, but Naive Bayes ignores these correlations.

Algorithms

Finding K (Elbow Method)

- If we choose $k = 1$ means the algorithm will be sensitive to outliers.
- If we choose $k = \text{all}$ (means the total number of data points in the training set), the majority class in the training set will always win.
- Always use k as an odd number.
- Elbow Method: We can also use an error plot or accuracy plot to find the most favorable k value.

Performance Metrics

Bias

Bias is the difference between the average prediction of the model and the expected value which it trying to predict.

- **High bias:** Underfitting, model cannot capture the data patterns, poor performance on training and test data.
- **Low bias:** Model fits training data well, Captures Complex Patterns, Risk of overfitting.

Variance

Variance is the variability in the model prediction, meaning how much the predictions will change if a different training data set is used.

- **High variance:** Overfitting, model captures noise, good performance on training but poor on test data i.e it fails to generalize to new, unseen data.
- **Low variance:** Model is too simple to capture the complexity of data, both training and test error are high.

Confusion Matrix

$$\begin{aligned}\text{Precision} &: \frac{\text{true positive}}{\text{all predicted positive}} = \frac{TP}{TP + FP} \\ \text{Recall/Sensitivity} &: \frac{\text{true positive}}{\text{all actual positives}} = \frac{TP}{TP + FN} \\ \text{Specificity} &: \frac{\text{true negative}}{\text{all actual negatives}} = \frac{TN}{TN + FP} \\ F_1\text{-score} &: \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}\end{aligned}$$

Errors

$$\text{Mean Absolute Error (MAE)} : \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

$$\text{Mean Squared Error (MSE)} : \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\text{Root Mean Squared Error (RMSE)} : \sqrt{\text{MSE}}$$

$$\text{R-squared} : 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

- MAE, MSE, and RMSE are often referred to as loss functions or error metrics because they quantify the error between predictions and true values.
- R^2 is an explanatory measure because it indicates how well the model fits the data.

Attribute Selection Metrics

Information Gain (ID3)

- All attributes are assumed to be categorical.
- Attribute with the highest information gain get selected.
- Entropy: A measure of uncertainty in the dataset

$$E(D) = \sum_{i=1}^m -p_i \log_2(p_i)$$

Where D : Dataset

C : The number of unique class labels in D

p_i : The proportion (probability) of class i in D .

- Weighted Entropy(A) = $\sum_{j=1}^v \frac{|A_j|}{|S|} \times E(A_j)$
- Gain(D, A) = $E(\text{Dataset}) - \text{Weighted Entropy}(A)$

Gain Ratio (C4.5)

- C4.5 (a successor of ID3) uses gain ratio to overcome the problem (normalization to information gain)
- SplitInfo(D, A) = $-\sum_{i=1}^v \frac{|A_i|}{|D|} \times \log_2 \left(\frac{|A_i|}{|D|} \right)$
- GainRatio(A) = $\frac{\text{Gain}(D, A)}{\text{SplitInfo}(D, A)}$

Gini Index (CART)

- CART creates a Binary Tree
- Gini(D) = $1 - \sum_{i=1}^m p_i^2$
- Gini(D, A) =

Problems with Attribute Selection Metrics

- **Information gain**: measure is biased towards attributes with a large number of values. (Solution: C4.5)
- **Gain Ration (C4.5)**: Tend to prefer unbalanced split in which one partition is much smaller than the other.
- **Gini Index (CART)**: Struggles with imbalanced datasets and can be sensitive to noise.

Linear Regression	Logistic Regression	SVM
Linear Regression predicts continuous values by modeling the relationship between a dependent variable and one or more independent variables using a straight line.	Predicts probabilities for binary outcomes, using the logistic function (sigmoid) to squeeze the output between 0 and 1. It is used for classification rather than regression.	The core idea of SVM is to find the best possible hyperplane ($N - 1$ dimensional) that separates the data points of different classes in an N -dimensional space.
- Simple model - Computationally efficient - Interpretability of the Output	- Overly-Simplistic - Linearity Assumption - Severely affected by Outliers	- Best for linearly separable data - Handles high-dimensional data efficiently - Adding new data doesn't affect the model - Memory efficient
- Overly-Simplistic - Linearity Assumption - Severely affected by Outliers	- Assumes linearity between inputs and log-odds - Not effective for complex or non-linear problems - Sensitive to outliers and multicollinearity	- Does not work well for large datasets. - Sensitive to noise and overlapping of target classes. - No probabilistic explanation for the predictions.

K-Means	KNN	Decision Tree
K-means clustering is an unsupervised machine learning algorithm used to partition N data points into a pre-defined number of K non-overlapping clusters.	Finds the k nearest neighbors and predict the class by the majority vote of the nearest neighbors. Its a lazy learning model i.e it requires no training.	Recursively splits data based on feature values to create a tree-like model, where nodes represent decisions and leaves represent outcomes or class labels.
- Simple and easy to understand & implement. - Computationally fast and efficient. - Guarantees convergence.	- No training period - Easy implementation - Adding new data doesn't affect the model	- Easy to interpret and visualize. - Handles numerical and categorical data. - Captures nonlinear relationships. - Minimal data preprocessing.
- Requires pre-specifying K (found by Elbow Method). - Sensitive to initial centroids and outliers. - Assumes clusters are spherical and similar in size.	- Does not work well for large datasets - Slower for high dimensions - Sensitive to noise - Feature scaling is mandatory	- Prone to overfitting. - Sensitive to small data changes. - Can be biased toward dominant features. - Less stable than ensemble methods.

Naive Bayes

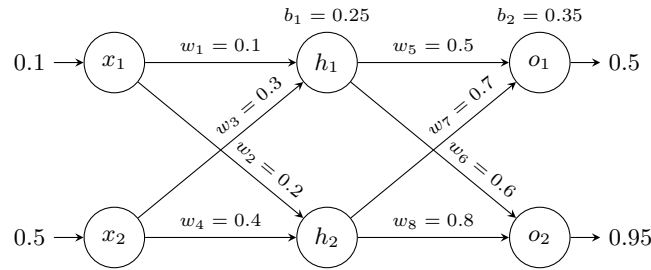
A probabilistic classifier based on Bayes' theorem, assuming all features are conditionally independent of each other, given the class.

Pros: Extremely fast, Easy to implement, Performs well on text classification

Cons: Unreliable probability estimates, Assumes features are independent , Zero Frequency Problem

Feed Forward Neural Network

Question: Assume that the neurons have a sigmoid activation function. Perform a forward pass on the given network and also calculate error.



For Hidden Layer:

$$h_{1(in)} = w_{11}x_1 + w_{21}x_2 + b_1$$

$$= 0.1 \times 0.1 + 0.3 \times 0.5 + 0.25 = 0.41$$

$$h_{1(out)} = \sigma(h_{1(in)}) = \frac{1}{1 + e^{-0.41}} = 0.601$$

$$h_{2(in)} = w_{12}x_1 + w_{22}x_2 + b_1$$

$$= 0.2 \times 0.1 + 0.4 \times 0.5 + 0.25 = 0.47$$

$$h_{2(out)} = \sigma(h_{2(in)}) = \frac{1}{1 + e^{-0.47}} = 0.615$$

For Output Layer:

$$o_{1(in)} = w_{51}(h_{1(out)}) + w_{71}(h_{2(out)}) + b_2$$

$$= 0.5 \times 0.601 + 0.7 \times 0.615 + 0.35 = 1.081$$

$$o_{1(out)} = \sigma(o_{1(in)}) = \frac{1}{1 + e^{-1.081}} = 0.746$$

$$o_{2(in)} = w_{52}(h_{1(out)}) + w_{72}(h_{2(out)}) + b_2$$

$$= 0.6 \times 0.601 + 0.8 \times 0.615 + 0.35 = 1.2026$$

$$o_{2(out)} = \sigma(o_{2(in)}) = \frac{1}{1 + e^{-1.2026}} = 0.769$$

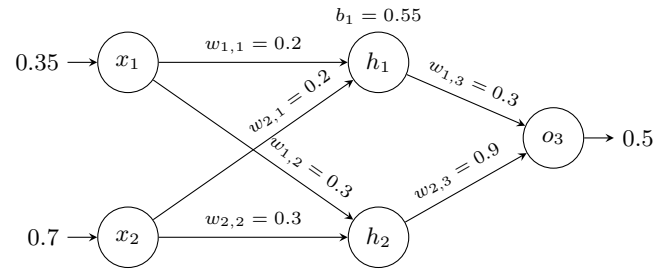
Error Calculation:

$$MSE = \frac{1}{2} [(t_1 - o_1)^2 + (t_2 - o_2)^2]$$

$$= \frac{1}{2} [(0.5 - 0.746)^2 + (0.95 - 0.769)^2]$$

$$= 0.0466$$

Question: Applying back propagation, update the weights $w_{1,1}$ and $w_{1,3}$ of the back propagation neural network shown in the following figure after one epoch. Sigmoid function is-used as an activation function. Learning rate $\alpha = 0.6$, actual output from node o_3 is 0.5, and bias for the hidden $b_1 = 0.55$:



For Hidden Layer:

$$h_{1(in)} = w_{11}x_1 + w_{21}x_2 + b_1$$

$$= 0.2 \times 0.35 + 0.2 \times 0.7 + 0.55 = 0.76$$

$$h_{1(out)} = \sigma(h_{1(in)}) = \frac{1}{1 + e^{-0.76}} = 0.68$$

$$h_{2(in)} = w_{12}x_1 + w_{22}x_2 + b_1$$

$$= 0.3 \times 0.35 + 0.3 \times 0.7 + 0.55 = 0.865$$

$$h_{2(out)} = \sigma(h_{2(in)}) = \frac{1}{1 + e^{-0.865}} = 0.703$$

For Output Layer:

$$o_{3(in)} = w_{13}(h_{1(out)}) + w_{23}(h_{2(out)}) + b_2$$

$$= 0.3 \times 0.68 + 0.9 \times 0.703 + 0 = 0.8367$$

$$o_{3(out)} = \sigma(o_{3(in)}) = \frac{1}{1 + e^{-0.8367}} = 0.697$$

Error Calculation:

$$MSE = \frac{1}{2} [(0.5 - 0.697)^2] = 0.0194$$

Information Gain

Construct the decision tree step by step, showing all calculations for Information Gain (using Entropy). Determine whether the email, where the length is long, does not contain links, and does not include attachments, is classified as spam.

Email Length	Contains Links	Contains Attachments	Spam?
Short	Yes	No	Yes
Long	No	Yes	No
Short	Yes	Yes	Yes
Long	Yes	No	No
Short	No	No	No

Entropy of entire dataset,

$$E(D) = E(2, 3) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.9709$$

Information Gain of Email Length,

$$E(Short) = E(2, 1) = -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} = 0.918$$

$$E(Long) = E(0, 2) = -0 \log_2 0 - \frac{2}{2} \log_2 \frac{2}{2} = 0$$

$$IG(D, Email) = 0.9709 - \left(\frac{3}{5} \times 0.918 + \frac{2}{5} \times 0 \right) = 0.4201$$

Information Gain of Contains Links,

$$E(Yes) = E(2, 1) = -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} = 0.918$$

$$E(No) = E(0, 2) = -0 \log_2 0 - \frac{2}{2} \log_2 \frac{2}{2} = 0$$

$$IG(D, Links) = 0.9709 - \left(\frac{3}{5} \times 0.918 + \frac{2}{5} \times 0 \right) = 0.4201$$

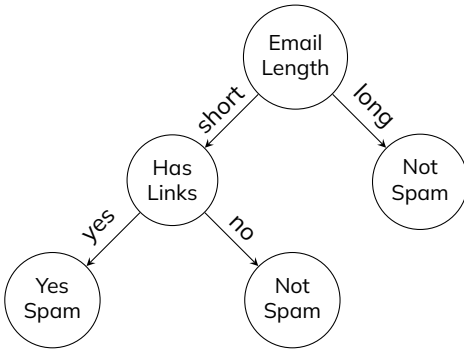
Information Gain of Contains Attachments,

$$E(Yes) = E(1, 1) = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1$$

$$E(No) = E(1, 2) = -\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} = 0.918$$

$$IG(D, Attach.) = 0.9709 - \left(\frac{2}{5} \times 1 + \frac{3}{5} \times 0.918 \right) = 0.0201$$

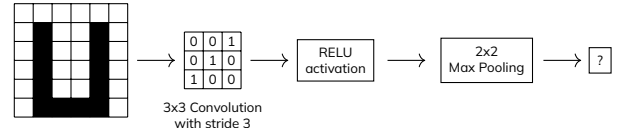
Since, $IG(D, Email Length) = IG(D, Contains Links) > IG(D, Contains Attachments)$, we can choose either Email Length or Contains Links as the root node.



If the email length is long, then it is classified as *Not Spam*.

CNN

Construct the output of each layer in the CNN illustrated in Figure 2, assuming white cells represent 0 and black cells represent 1:



Solution:

Converting image to matrix form:

0	0	0	0	0	0
0	1	0	0	1	0
0	1	0	0	1	0
0	1	0	0	1	0
0	1	0	0	1	0
0	1	1	1	1	0

After 3x3 Convolution with stride 3:

$$C_{1,1} = (0 \cdot 0) + (0 \cdot 0) + (0 \cdot 1) + (0 \cdot 0) + (1 \cdot 1) + (0 \cdot 0) + (0 \cdot 1) + (1 \cdot 0) + (0 \cdot 0) = 1$$

$$C_{1,2} = (0 \cdot 0) + (0 \cdot 0) + (0 \cdot 1) + (0 \cdot 0) + (1 \cdot 1) + (0 \cdot 0) + (0 \cdot 1) + (1 \cdot 0) + (0 \cdot 0) = 1$$

$$C_{2,1} = (0 \cdot 0) + (1 \cdot 0) + (0 \cdot 1) + (0 \cdot 0) + (1 \cdot 1) + (0 \cdot 0) + (0 \cdot 1) + (1 \cdot 0) + (1 \cdot 0) = 1$$

$$C_{2,2} = (0 \cdot 0) + (1 \cdot 0) + (0 \cdot 1) + (0 \cdot 0) + (1 \cdot 1) + (0 \cdot 0) + (1 \cdot 1) + (1 \cdot 0) + (0 \cdot 0) = 2$$

1	1
1	2

After Relu activation:

1	1
1	2

After 2x2 max pooling:

2
